

Занятие 4

Реализация ETL. Очистка данных

4.1. Цели и задачи работы

Цель работы: реализовать спроектированный на прошлом занятии ETL-процесс на языке Python, выполнить очистку и преобразование данных, подготовить очищенный датасет для загрузки в хранилище.

Задачи работы:

1. Реализовать извлечение данных из исходных источников.
2. Выполнить все запланированные трансформации.
3. Обработать пропуски, дубликаты и выбросы.
4. Сохранить очищенные данные в промежуточный файл.
5. Создать отчёт о выполненной очистке с метриками качества.

4.2. Теоретические сведения

4.2.1. Основные методы очистки в Pandas

Операция	Метод Pandas	Описание
Удаление дубликатов	<code>df.drop_duplicates()</code>	Удаляет полностью дублирующиеся строки
Удаление пропусков	<code>df.dropna()</code>	Удаляет строки с пропусками
Заполнение пропусков	<code>df.fillna(value)</code>	Заполняет пропуски указанным значением
Замена значений	<code>df.replace()</code>	Заменяет одни значения другими
Приведение типов	<code>df.astype()</code>	Изменяет тип данных колонки
Фильтрация	<code>df[df['col'] > 0]</code>	Оставляет строки, удовлетворяющие условию
Применение функции	<code>df.apply()</code>	Применяет функцию к каждой строке/элементу

4.2.2. Обработка пропусков

Стратегии заполнения пропусков:

Стратегия	Метод	Когда применять
Константой	<code>fillna(0)</code>	Для числовых полей, где 0 имеет смысл
Средним	<code>fillna(df['col'].mean())</code>	Для нормально распределённых данных
Медианой	<code>fillna(df['col'].median())</code>	Для данных с выбросами
Модой	<code>fillna(df['col'].mode()[0])</code>	Для категориальных данных

Предыдущим значением	<code>fillna(method='ffill')</code>	Для временных рядов
Интерполяцией	<code>interpolate()</code>	Для плавных временных рядов
Удаление	<code>dropna()</code>	Если пропусков мало (<5%)

4.2.3. Обработка выбросов

Методы обнаружения выбросов:

1. Метод IQR (межквартильного размаха):

```
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = df[(df['col'] < lower_bound) | (df['col'] > upper_bound)]
```

2. Метод Z-оценки:

```
from scipy import stats
z_scores = np.abs(stats.zscore(df['col']))
outliers = df[z_scores > 3]
```

Стратегии обработки выбросов:

- Удаление (если выбросов мало, и они ошибочны)
- Винзоризация (замена на граничные значения)
- Логарифмическое преобразование
- Оставить как есть (если выбросы важны для анализа)

4.3. Порядок выполнения работы

Шаг 1. Подготовка среды и загрузка данных

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Загрузка сырых данных
df = pd.read_csv('data_raw.csv')
print(f"Исходный размер: {df.shape}")
```

Шаг 2. Реализация трансформаций

Выполните все запланированные трансформации в логической последовательности:

- Приведение типов
- Удаление дубликатов
- Обработка пропусков

- Обработка выбросов
- Создание новых признаков
- Нормализация текстовых полей

Шаг 3. Валидация результатов

```
# Проверка, что пропусков не осталось
missing_after = df.isnull().sum()
print("Пропуски после обработки:")
print(missing_after[missing_after > 0])

# Сравнение статистик до/после
print("\nСтатистика до обработки:")
print(pd.read_csv('data_raw.csv').describe())
print("\nСтатистика после обработки:")
print(df.describe())

# Визуализация результатов
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df_raw = pd.read_csv('data_raw.csv')
if 'age' in df_raw.columns and 'age' in df.columns:
    axes[0].hist(df_raw['age'].dropna(), bins=30, alpha=0.5, label='До')
    axes[0].hist(df['age'], bins=30, alpha=0.5, label='После')
    axes[0].set_title('Распределение возраста')
    axes[0].legend()
plt.show()
```

Шаг 4. Сохранение очищенных данных

```
# Сохранение в CSV
df.to_csv('data_cleaned.csv', index=False)
print("Очищенные данные сохранены в data_cleaned.csv")

# Сохранение отчёта о качестве
quality_report = pd.DataFrame({
    'Поле': df.columns,
    'Тип': df.dtypes.values,
    'Уникальных': df.nunique().values,
    'Пропусков': df.isnull().sum().values,
    'Пропусков_%': (df.isnull().sum() / len(df) * 100).values
})
quality_report.to_csv('quality_report.csv', index=False)
print("Отчёт о качестве сохранён в quality_report.csv")
```

4.4. Структура отчёта

Раздел 1. Цель работы

Раздел 2. Реализация ETL-процесса

- Код основных этапов с комментариями
- Таблица выполненных трансформаций

Раздел 3. Статистика очистки

- Количество удалённых дубликатов
- Доля обработанных пропусков по полям

- Количество обработанных выбросов

Раздел 4. Сравнение «до» и «после»

- Таблица сравнения статистик
- Визуализации (гистограммы, box plots)

Раздел 5. Выводы о качестве полученных данных

4.5. Контрольные вопросы

1. Какие стратегии заполнения пропусков вы использовали и почему?
2. Как вы обрабатывали выбросы и почему выбрали именно этот метод?
3. Какие новые признаки были созданы и для чего они нужны?
4. Как изменилось распределение данных после обработки?
5. Достаточно ли очищенных данных для построения хранилища?

4.6. Литература

1. Маккинни, У. Python и анализ данных (см. полное описание в занятии 2) — главы о работе с пропусками, дубликатами и трансформацией в pandas.
2. Очистка данных: проблемы и современные подходы // Хабр. URL: <https://habr.com/ru/articles/548220/> (дата обращения: 10.07.2026). Систематизация типов проблем с данными.
3. Проблемы, методы и вызовы комплексной очистки данных // Хабр. URL: <https://habr.com/ru/articles/548356/> (дата обращения: 10.07.2026). Классификация аномалий и критериев качества данных.
4. Основы очистки данных в data science // Хабр. URL: <https://habr.com/ru/articles/875662/> (дата обращения: 10.07.2026). Практика работы с пропусками, выбросами, форматированием.
5. Очистка данных перед загрузкой в хранилище. Подробное руководство с техническими деталями // Хабр. URL: <https://habr.com/ru/articles/794036/> (дата обращения: 10.07.2026). Связывает очистку с загрузкой в DWH, с примерами кода.